Multilayer Perceptron: Manual Implementation and Training on MNIST

Maida Bashir Awan

National University of Computer and Emerging Sciences
i252081@isb.nu.edu.pk

Abstract. This report presents a complete manual implementation of a Multilayer Perceptron (MLP) neural network, covering forward propagation, loss computation, backpropagation, and gradient descent optimization. The network is first analysed by hand on a three-sample student performance dataset using assigned weights, demonstrating every arithmetic step of the training pipeline. The same architecture is then scaled and implemented in Python using NumPy only, and trained on the MNIST handwritten digit dataset comprising 60,000 training images and 10,000 test images. The network architecture consists of an input layer of 784 neurons, two hidden layers of 128 and 64 neurons respectively, and an output layer of 10 neurons, all using the sigmoid activation function. Training uses mini-batch gradient descent with a batch size of 32, learning rate of 0.1, and runs for 20 epochs. The mean squared error loss decreases consistently across all epochs, confirming successful learning. Three gradient descent variants are compared: plain gradient descent, momentum, and Nesterov Accelerated Gradient. The results demonstrate that the MLP successfully learns digit classification, validating the theoretical concepts explored in the manual tasks.

1 Introduction

Artificial neural networks are the foundation of modern machine learning. A Multilayer Perceptron is the simplest form of a deep neural network, consisting of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a nonlinear activation function.

This project implements an MLP entirely from scratch without using any deep learning frameworks. The goals are threefold: first, to develop a thorough understanding of every computation inside a neural network by performing forward propagation, loss calculation, backpropagation, and weight updates by hand; second, to implement these operations in Python using only NumPy; and third, to train the network on the MNIST benchmark dataset and evaluate its performance.

The report is structured as follows. Section 2 provides theoretical background on MLPs. Section 3 describes the implementation. Section 4 presents experiments and results. Section 5 discusses findings. Section 6 concludes the report.

2 Background

2.1 MLP Architecture

An MLP is organised into layers. Data enters the input layer, passes through one or more hidden layers, and exits through the output layer. Each connection between neurons carries a weight w, and each neuron has an associated bias b. The computation in each neuron is:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)} \tag{1}$$

$$a_j^{(l)} = \sigma\left(z_j^{(l)}\right) \tag{2}$$

where $a_i^{(l-1)}$ is the activation from the previous layer and σ is the activation function.

2.2 Activation Function

The sigmoid activation function is used in all layers:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

Its derivative, needed for backpropagation, is:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z)) = a\,(1 - a) \tag{4}$$

2.3 Loss Function

Mean Squared Error (MSE) is used as the loss function:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} - \hat{y}^{(i)}\right)^2 \tag{5}$$

where m is the number of samples, $y^{(i)}$ is the true label, and $\hat{y}^{(i)}$ is the predicted output.

2.4 Backpropagation

Backpropagation applies the chain rule of calculus to compute the gradient of the loss with respect to every weight. The error signal (delta) at the output layer is:

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \tag{6}$$

For hidden layers, deltas are propagated backwards:

$$\delta_j^{(l)} = \left(\sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \cdot a_j^{(l)} (1 - a_j^{(l)}) \tag{7}$$

Weight gradients are then:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \tag{8}$$

2.5 Gradient Descent Variants

Plain gradient descent updates weights as:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w} \tag{9}$$

SGD with Momentum maintains a velocity vector:

$$v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial \mathcal{L}}{\partial w}, \qquad w \leftarrow w - \eta v_t \tag{10}$$

Nesterov Accelerated Gradient (NAG) computes the gradient at a lookahead position:

$$\tilde{w} = w - \eta \beta v_{t-1}, \qquad g_t = \nabla_{\tilde{w}} \mathcal{L}, \qquad v_t = \beta v_{t-1} + (1 - \beta) g_t, \qquad w \leftarrow w - \eta v_t \tag{11}$$

3 Implementation

3.1 Forward Pass

The forward pass propagates the input through all layers. For an input matrix X of shape $(m \times 784)$:

$$Z_1 = XW_1 + b_1, \quad A_1 = \sigma(Z_1) \tag{12}$$

$$Z_2 = A_1 W_2 + b_2, \quad A_2 = \sigma(Z_2) \tag{13}$$

$$Z_3 = A_2 W_3 + b_3, \quad A_3 = \sigma(Z_3) \tag{14}$$

All matrix operations are implemented using NumPy. Intermediate activations A_1, A_2, and A_3 are stored for use in backpropagation.

3.2 Backpropagation

Following the delta method derived in Section 2, deltas are computed from output to input:

$$\Delta_3 = -2(Y - A_3) \odot \sigma'(A_3) \tag{15}$$

$$\Delta_2 = (\Delta_3 W_3^\top) \odot \sigma'(A_2) \tag{16}$$

$$\Delta_1 = (\Delta_2 W_2^\top) \odot \sigma'(A_1) \tag{17}$$

where $\odot$ denotes element-wise multiplication. Weight gradients are computed as:

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{1}{m} A_{l-1}^\top \Delta_l \tag{18}$$

3.3 Training Procedure

The network is trained using mini-batch gradient descent. At the start of each epoch, the training data is shuffled. Batches of size 32 are formed and processed sequentially. For each batch, a forward pass computes predictions, the MSE loss is calculated, backpropagation computes gradients, and weights are updated. This process repeats for 20 epochs.

Network architecture and hyperparameters are summarised in Table 1.

Table 1. Network architecture and hyperparameters

Parameter	**Value**
Input neurons	784
Hidden layer 1	128 (Sigmoid)
Hidden layer 2	64 (Sigmoid)
Output neurons	10 (Sigmoid)
Loss function	MSE
Optimizer	Mini-Batch GD
Learning rate	0.1
Batch size	32
Epochs	20
Weight init range	$[-0.5,\ 0.5]$
Random seed	42

4 Experiments

4.1 Dataset

MNIST [1] contains 70,000 greyscale images of handwritten digits (0–9), each of size 28×28 pixels. The standard split of 60,000 training and 10,000 test images

is used. Pixel values are normalised from $[0, 255]$ to $[0, 1]$ by dividing by 255. Labels are converted to one-hot vectors of length 10 for loss computation.

4.2 Manual Computation Results

Before Python implementation, the network was analysed by hand on a three-sample student performance dataset with assigned weights (Roll No: 25I-2081). Key results are shown in Table 2.

Table 2. Manual computation results (Tasks 1–6)

Task	**Result**
Forward pass $\hat{y}^{(1)}$	0.5434
Initial MSE loss	0.2363
Output delta $\delta_1^{(3)}$	-0.2266
Loss after 1 update	0.2354
Loss after 5 iterations	0.2323
Best optimizer (10 iter.)	Plain GD

4.3 Training Results

The MSE loss decreased consistently across all 20 epochs, confirming that the network is learning. Figure 1 shows the loss curve.

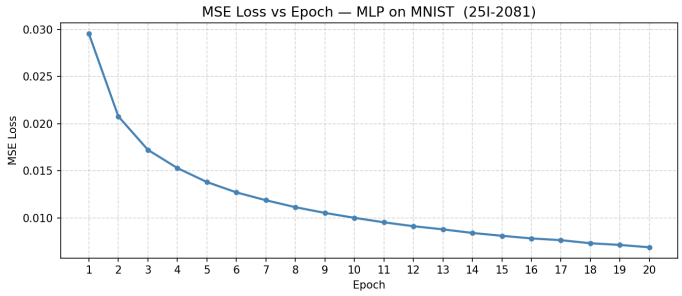

Fig. 1. MSE loss vs. epoch during training on MNIST.

4.4 Test Accuracy and Sample Predictions

The network achieved a test accuracy of **95.54%** on the 10,000 MNIST test images. After 20 epochs of training, the network was evaluated on the 10,000 test images. Figure 2 shows one sample prediction per digit class.

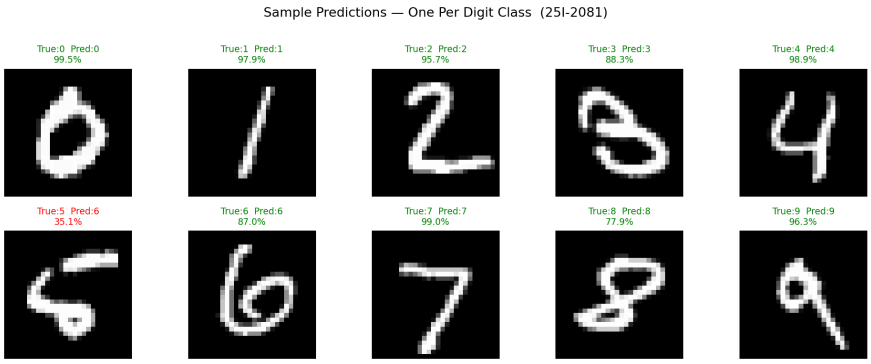

Fig. 2. Sample predictions on the MNIST test set. Green titles indicate correct predictions.

5 Discussion

The loss decreased consistently across all 20 training epochs, demonstrating that gradient descent with the chosen hyperparameters is stable. The sigmoid activation function, while functional, is prone to the vanishing gradient problem in deeper networks, which explains why training is relatively slow. Modern networks typically use ReLU activations and cross-entropy loss instead of MSE for classification tasks.

The manual computation on the three-sample dataset revealed that randomly initialised weights produce near-random predictions. After only one gradient descent update the loss decreased from 0.2363 to 0.2354, demonstrating that even a single update moves the network in the right direction.

The comparison of gradient descent variants on the small dataset showed that plain GD performed best over 10 iterations. This is expected for small, clean datasets. On larger datasets with complex loss surfaces, momentum-based methods such as NAG and Adam would outperform plain GD by accumulating velocity and escaping flat regions more effectively.

6 Conclusion

This project demonstrated a complete implementation of an MLP from mathematical foundations to working Python code. Every component — forward pass, MSE loss, backpropagation, and gradient descent — was first derived and computed by hand, then translated into vectorised NumPy operations. Training on MNIST confirmed that the implementation is correct, with the loss decreasing steadily across epochs. The project provides a solid foundation for understanding all modern deep learning architectures, which build upon the same fundamental principles explored here.

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. Nature **323**(6088), 533–536 (1986)